Concept of Type

We tend to classify our friends. Is it not? We will say that fellow is a “kudumi”. We classify a few as sportsmen. A few as musicians and so on. All those in the same group exhibit some common behaviour.

This concept of grouping has come from Mathematics – concept of Set. We study number line. We start with set of natural numbers. Apply some operations on it. Show that some do not result in a member of the same set – we say that this is set is not closed for this operation. We then extend the set – to set of integer, then set of rational numbers – the set of irrational numbers and then set of real and last set of complex numbers. My teacher had told me in my PU days that there is a theorem which says that there can be no number system beyond the complex number set.

Let us look at set of integers in Mathematics.

It is closed for addition and multiplication
25 + 34 is 59
How do we add?

- Add unit digits and then digits in tenth place and so on along with the carry from the earlier stage
- Add row wise
- Use fingers and toes
- Use tally marks
- Convert to some other base, add and convert the result back to decimal
- Use Vedic Math or Trachtenberg method

Will the result be different?
Should we learn how to add if the integers have different values?

The beauty of this grouping is that we can treat them uniformly and we can the define the results of operations on them without explicitly saying how to carry these operations.

- An integer is a type
- Represented in a programming language normally as a subset
- A language will specify the smallest and the biggest integers in its implementation.
- **A type indicates**
  - **What values are legal**
  - **What we can do with them**
  - **Does not say how we can do them**
  - **Allows us to treat almost all values of that type uniformly**

- Type of a variable in ‘C’ has to be declared in the beginning
- Once declared, cannot be changed.
- Type of a variable is determined at compile time
- In Python,
  - type is associated with a value.
  - A variable gets a type based on type of value assigned.
  - Type is dynamic.

Can a language support all possible types of the real world?

- A car is a type.
- Supports the following operations
    - Start
    - Stop
    - Accelerate
    - Brake
    - Steer
- Once you have learnt driving a car, you can (almost) drive any other car
- Other types of real world : fan, mobile phone, browser, laptop, novel, dream, …
- Can a language support all possible types of the real world?

The answer is a big NO.

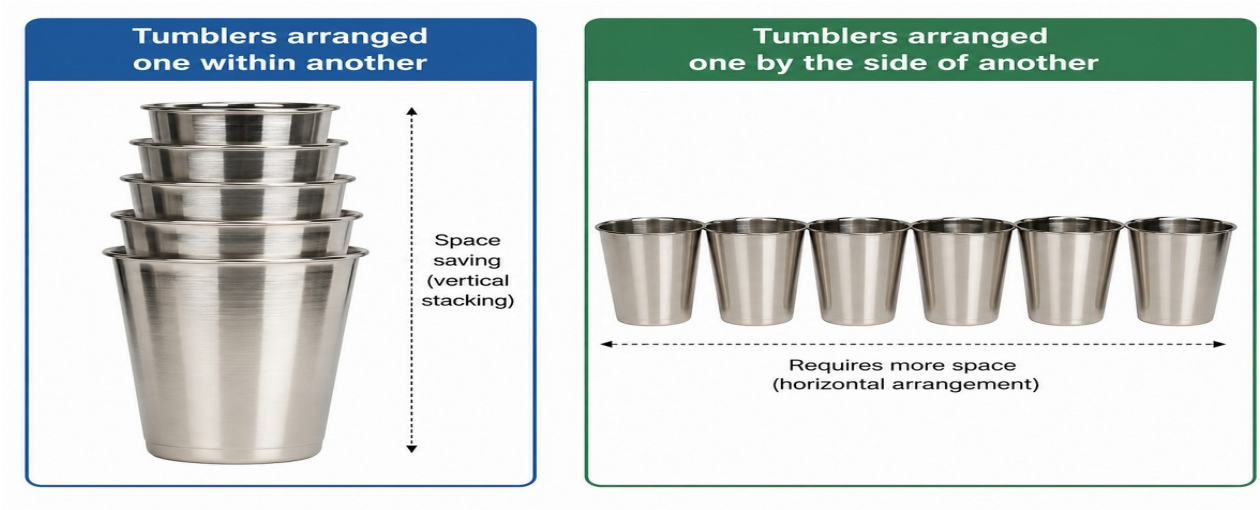
An interface is sacred – cannot be changed
An implementation – can be changed .
In a car, the braking mechanism can be changed from mechanical to hydraulic system.
Steering can be changed from manual to power steering.

Can we change the interface? When you press the brake, the vehicle accelerates? When you turn the steering to the left, the vehicle goes to the right? Can you imagine what could happen?

What should the language support?

- A Language should support a way of creating our own type
- **Abstract Data Type (ADT)**
    - Like a type (int, double, …) in a language, this supports
        - A few operations
        - Specifies what they do and not how they do
        - **Specifies the interface and not the implementation**
        - Both the user and the implementor depend on the interface
        - Interface once decided is sacred; cannot be changed
        - Implementation can be changed without affecting the user
        - This is called inversion of dependency
    - If A depends on B, any change to A would require change to B.
    - If both A and B depend on what does not change, change to A will not affect B.

Data Structure:
Is an interface
Has number of elements arranged in some particular way to support a few operations efficiently
Is constrained by time and space.



In the above example, tumblers arranged in two different ways. None of them is wrong. The first one is saving space and only one person can pick up a tumbler at a time. The arrangement on the right requires more space, but more than one access at the same time.
Should the tumblers be placed mouth up or mouth done. I do not know have an answer!

In this arrangement of books, one of the left allows easy access to any book whereas the one on the right requires that you have pull the book with a lot of effort if it is towards the bottom. The second may save space and allow more books to be placed.



None of them is right or wrong. You may select one of these arrangements based on your requirement.

List :

- Is a data structure
- Has zero or more components
- Each element has an ordinal position
- Cannot directly access the element based on position
- Sequential access
- No random or direct access
- Insertion and deletion once the position is known requires
- A few operations

- **Interface** :
    - Create
    - Destroy
    - Insert
    - Delete
    - …
- **Implement** using
    - A linked list
    - An Array